

TypeScript Full Revision Notes: Theory & Code

Full Stack Developer Prep

1 Core Fundamentals

- **TypeScript:** A typed superset of JavaScript. Compiles to plain JS via `tsc`. Types are stripped during compilation and have zero runtime performance impact.
- **Basic Types:** `string`, `number`, `boolean`, arrays (`number[]`), tuples (`[string, number]`).
- **Special Types:**
 - `any`: Disables type checking completely. (Avoid using).
 - `unknown`: A type-safe alternative to `any`. Forces you to perform a type check (narrowing) before usage.
 - `void`: Used for functions that return nothing.
 - `never`: Used for functions that never finish executing (e.g., infinite loops, throwing errors).

2 Interfaces vs. Type Aliases (Highly Tested)

- **Interface:** Used mainly for defining object/class shapes.
 - **Declaration Merging:** If you declare the same interface twice, TS merges them. Great for extending global libraries.
 - **Extending:** Uses the `extends` keyword.
- **Type Alias:** Can define object shapes, but also primitives, unions, and tuples.
 - **Closed:** Cannot be reopened or merged once defined.
 - **Extending:** Combines via Intersection (`&`).
- **Golden Rule:** Use `interface` for object shapes and public APIs; use `type` for unions, intersections, and primitive aliases.

```
1 interface User {  
2   readonly id: number; // Cannot be reassigned  
3   name: string;  
4   email: string;  
5   role?: string;      // Optional property  
6 }
```

Listing 1: Interface Shapes and Optional/Readonly Properties

3 Advanced Typing

- **Union Types (|)**: Variable can be one of multiple distinct types (OR).
- **Type Assertions**: Using `as Type` to override the compiler's inference. Dangerous because it assumes you know better than the compiler without runtime checks.
- **Type Guards**: Standard JS logic used to safely narrow down broad types.
 - `typeof`: Checks primitives (e.g., `typeof val === 'string'`).
 - `instanceof`: Checks class instances.
 - `in`: Checks for a property in an object (e.g., `'email' in user`).
 - **Custom Predicates**: Functions returning `arg is Type`.

```
1 // The "status" property acts as the discriminator literal
2 interface SuccessResponse {
3   status: "success";
4   data: { id: number; name: string };
5 }
6
7 interface ErrorResponse {
8   status: "error";
9   errorMessage: string;
10 }
11
12 type ApiResponse = SuccessResponse | ErrorResponse;
13
14 function handleApi(response: ApiResponse) {
15   // TypeScript narrows the type based on the 'status' check
16   if (response.status === "success") {
17     console.log("Data received:", response.data.name); // Strictly
18     SuccessResponse
19   } else {
20     console.error("Error occurred:", response.errorMessage); //
21     Strictly ErrorResponse
22   }
23 }
```

Listing 2: Discriminated Unions (API Response Handler)

```
1 type Fish = { swim: () => void };
2 type Bird = { fly: () => void };
3
4 // The return signature 'pet is Fish' tells TS how to narrow the type
5 function isFish(pet: Fish | Bird): pet is Fish {
6   return (pet as Fish).swim !== undefined;
7 }
8
9 function move(pet: Fish | Bird) {
10   if (isFish(pet)) {
11     pet.swim(); // TS knows pet is Fish
12   } else {
13     pet.fly(); // TS knows pet is Bird
14   }
15 }
```

Listing 3: Custom Type Guards using the 'is' keyword

4 Generics & Utility Types

- **Generics** (<T>): Variables for types. They allow creation of reusable components without sacrificing type safety or resorting to **any**.
- **Constraints**: Limiting a generic using **extends** (e.g., <T extends { length: number }>).
- **Utility Types**:
 - **Partial**<T>: Makes all properties in T optional (?).
 - **Pick**<T, K>: Selects specific properties K from T.
 - **Omit**<T, K>: Removes specific properties K from T.
 - **Record**<K, T>: Creates an object type mapping keys of type K to values of type T.

```
1 // Using the User interface from Section 2
2
3 // 1. Partial: Makes all properties optional
4 type UpdateUserPayload = Partial<User>;
5
6 // 2. Pick: Selects specific properties
7 type UserPreview = Pick<User, "id" | "name">;
8
9 // 3. Omit: Removes specific properties
10 type UserWithoutEmail = Omit<User, "email">;
```

Listing 4: Built-in Utility Types

```
1 // Record<string, number> enforces that keys are strings and values are
   numbers
2 function getWordFrequency(text: string): Record<string, number> {
3   const words = text.toLowerCase().split(/\W+/).filter(Boolean);
4   const frequency: Record<string, number> = {};
5
6   for (const word of words) {
7     if (frequency[word]) {
8       frequency[word]++;
9     } else {
10      frequency[word] = 1;
11    }
12  }
13  return frequency;
14 }
```

Listing 5: Using Record<K

5 Classes & OOP

- **Access Modifiers**:
 - **public** (Default): Accessible anywhere.
 - **private**: Accessible ONLY within the class itself (compile-time check).

- **protected**: Accessible within the class and its subclasses.
- **Parameter Properties**: Shorthand to declare and initialize properties directly in the constructor parameters.
- **Abstract Classes**: Base classes that cannot be directly instantiated via `new`. Require subclasses to implement their **abstract** methods.
- **Implements**: Keyword ensuring a class strictly adheres to an **interface**.

```

1 // 1. Define the constraint
2 interface HasId {
3   id: string;
4 }
5
6 // 2. Class uses Generic <T> that must satisfy the HasId constraint
7 class DataStore<T extends HasId> {
8   // 3. Using private access modifier
9   private items: T[] = [];
10
11   public addItem(item: T): void {
12     this.items.push(item);
13   }
14
15   public getItem(id: string): T | undefined {
16     return this.items.find(item => item.id === id);
17   }
18
19   public getAllItems(): T[] {
20     return this.items;
21   }
22 }
23
24 const userStore = new DataStore<{ id: string; name: string }>();
25 userStore.addItem({ id: "u1", name: "Alice" }); // Valid
26 // userStore.addItem({ name: "Bob" }); // ERROR: Property 'id' is
    missing

```

Listing 6: Combining Classes with Generic Constraints

6 Modules & tsconfig.json

- **Modules**: Use named exports (`export const X`) for better refactoring safety, or default exports (`export default X`).
- **tsconfig.json Key Flags**:
 - **target**: The JS version to output (e.g., ES5, ES2022).
 - **module**: The module system for output (e.g., CommonJS, ESNext).
 - **strict**: **Always true**. Enables rigorous type checking features like `strictNullChecks`.
 - **outDir**: Destination folder for compiled JavaScript.
 - **lib**: Default typing libraries to include (e.g., `["DOM"]` for browser APIs).